// DEEP DIVE

SIEM & Log Analysis

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How raw logs from thousands of sources become a single actionable alert — the collection-to-correlation pipeline, Wazuh and Splunk architecture side by side, and how to write a detection rule that actually holds up.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	What a SIEM does, and the log pipeline end to end
3 — 4	Wazuh architecture and Splunk architecture, compared
5 — 6	Correlation rules/detection logic, and log normalization
7 — 9	Alert tuning, a worked correlation rule, and reference

How to use this guide: read start to finish for the full picture, or jump to section 8 for the worked rule-building example. Every diagram is numbered and referenced directly in the surrounding text.

© 2026 · Swastik Sagar · deep dive · free reference

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What a SIEM Actually Does

1.1 THREE JOBS IN ONE PLATFORM

A **SIEM (Security Information and Event Management)** platform combines three distinct capabilities that used to require separate tools: **Security Information Management** — long-term log storage, search, and reporting; **Security Event Management** — real-time monitoring and correlation of events as they arrive; and alerting on the results of both. The core value isn't any single log source — it's the ability to correlate events *across* many sources (firewalls, endpoints, servers, cloud services) that individually look unremarkable, but together reveal an attack.

> **A SIEM doesn't generate security data — it aggregates and makes sense of it.** Every log a SIEM ingests originates somewhere else entirely (a Linux host's auth.log, covered in the Linux for SOC Analysts guide; a Windows Event Log, covered in the Windows Event Log Reference guide; a firewall). The SIEM's value is entirely in collecting all of it centrally and finding patterns no single source reveals alone.

1.2 WHY CENTRALIZATION SPECIFICALLY MATTERS

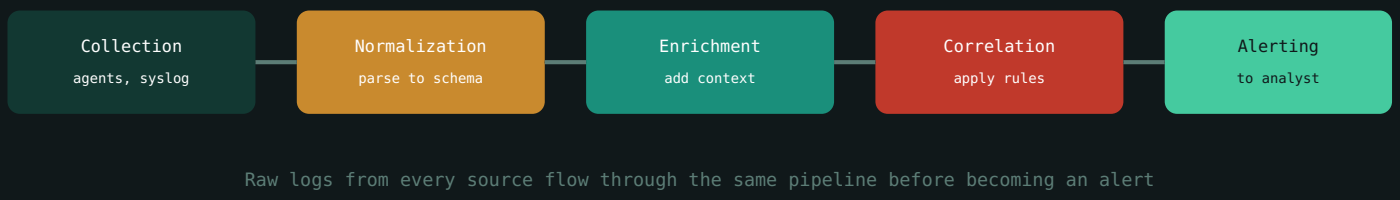
- **Cross-source correlation** — a failed login on a firewall plus a successful login on a server plus an unusual file access, each unremarkable alone, becomes an obvious attack pattern when viewed together.
- **Tamper resistance** — logs shipped to a central SIEM before an attacker gains full host control are far harder to erase than logs left sitting only on the compromised host (see the Linux for SOC Analysts guide, Section 5.3).
- **Retention & retrospective search** — when a new threat or IOC (indicator of compromise) becomes known, a SIEM lets you search historical logs for it, not just watch for it going forward.
- **Compliance & reporting** — many regulatory frameworks require centralized logging and defined retention periods that a SIEM is built to satisfy.

// SECTION 2

The Log Pipeline: Collection to Correlation

2.1 THE FIVE STAGES

Every SIEM, regardless of specific product, moves data through roughly the same pipeline stages before it ever becomes an alert an analyst sees.



Raw logs from every source flow through the same pipeline before becoming an alert

Fig 2.1 — The SIEM pipeline: raw logs are collected, normalized into a common schema, enriched with context, correlated against rules, and only then alerted.

2.2 STAGE DETAIL

STAGE	WHAT HAPPENS
Collection	Agents (Section 3) or syslog forwarding bring raw log data from every source into the platform
Normalization	Different log formats are parsed into a common schema (Section 6), so a "failed login" means the same thing regardless of source
Enrichment	Additional context is attached — geolocation for an IP, a threat intelligence match, asset criticality
Correlation	Rules (Section 5) examine normalized, enriched events — individually or across multiple events — for patterns matching known attack behavior
Alerting	A matched correlation rule generates an alert for an analyst, ideally with enough context to begin triage immediately

Every later section in this guide maps onto one of these five stages — Sections 3–4 cover collection architecture specifically, Section 6 covers normalization in depth, and Section 5 covers correlation.

// SECTION 3

Wazuh Architecture

3.1 AGENT-BASED, OPEN SOURCE

Wazuh is a free, open-source SIEM/XDR platform built around lightweight agents installed directly on monitored hosts. Each agent collects logs, monitors file integrity, and can perform local detection before ever sending data upstream.

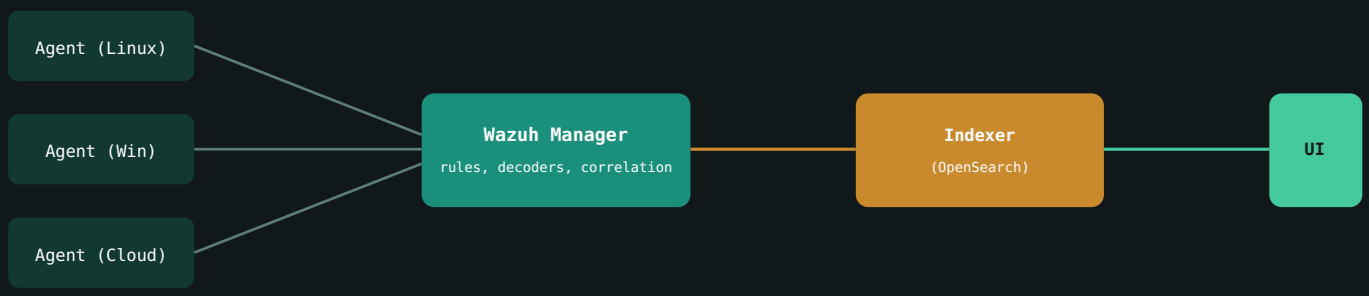


Fig 3.1 — Wazuh's architecture: lightweight agents forward data to a central manager, which applies detection rules before indexing results for search and dashboards.

3.2 CORE COMPONENTS

COMPONENT	ROLE
Agent	Installed on each monitored host; collects logs, checks file integrity, runs local rootkit/vulnerability checks
Manager	Receives agent data, applies decoders and correlation rules (Section 5), generates alerts
Indexer	Stores and indexes processed data (built on OpenSearch) for search and retention
Dashboard	The web UI analysts actually use to search, investigate, and view alerts

Wazuh does more than log collection. Because its agents run directly on the host, Wazuh also natively performs file integrity monitoring, vulnerability detection, and configuration assessment — capabilities that in a pure log-only SIEM would need entirely separate tooling.

// SECTION 4

Splunk Architecture

4.1 FORWARDER-BASED, COMMERCIAL

Splunk is a widely deployed commercial platform (with a free tier for smaller volumes) built around a similar but distinctly named component model: lightweight **forwarders** instead of Wazuh's agents, an **indexer** that also handles processing, and a **search head** as the primary analyst interface.

COMPONENT	ROLE
Universal Forwarder	A lightweight agent installed on hosts, forwarding raw data with minimal local processing
Indexer	Receives forwarded data, parses and indexes it, stores it for search — the workhorse of the platform
Search Head	Where analysts run searches (using Splunk's own SPL query language) and view dashboards
Heavy Forwarder	An optional forwarder variant that can parse/filter data before sending it on, reducing indexer load

4.2 WAZUH VS SPLUNK: WHERE THEY ACTUALLY DIFFER

ASPECT	WAZUH	SPLUNK
Licensing	Free, open source	Commercial, licensed by data volume ingested
Host-level capability	Built-in FIM, vulnerability, and config checks	Primarily log collection — extended capability via apps/add-ons
Query language	OpenSearch query DSL	SPL (Search Processing Language) — powerful, Splunk-specific
Typical fit	Cost-sensitive deployments, smaller teams, host-centric monitoring	Large enterprises with budget for extensive third-party integration/apps

The pipeline concept from Section 2 is identical either way. Despite different component names — agent vs forwarder, manager vs indexer — both platforms move data through the exact same collection → normalization → enrichment → correlation → alerting pipeline. Learning one deeply makes the other's architecture recognizable almost immediately.

// SECTION 5

Correlation Rules & Detection Logic

5.1 WHAT A CORRELATION RULE ACTUALLY IS

A **correlation rule** defines a pattern of events — often across multiple log sources or a time window — that, when matched, indicates something worth an analyst's attention. Rules range from very simple (a single event, like "a firewall blocked a connection from a known-malicious IP") to genuinely correlated (multiple distinct events, from different sources, occurring together or in sequence).

5.2 SINGLE-EVENT VS MULTI-EVENT RULES

TYPE	EXAMPLE	COMPLEXITY
Single-event	One Windows Event ID 4720 (account created) fires an alert	Simple — direct match
Threshold-based	5 failed logins (Event ID 4625, see the Windows Event Log Reference guide) from one source within 2 minutes	Moderate — counting within a window
Multi-source correlation	A failed VPN login, followed by a successful login from a new geography, followed by unusual file access	High — sequence across multiple, distinct sources

5.3 DETECTION LOGIC: WHAT MAKES A RULE GOOD

- **Specificity** — the rule should match the actual malicious pattern, not just anything superficially similar (a major driver of false positives, Section 7).
- **Appropriate time windows** — too short a window misses slow, deliberate attacks; too long a window correlates unrelated events together.
- **Context awareness** — a rule that accounts for asset criticality or known-good exceptions (a scheduled admin script, for instance) produces far more actionable alerts than one applied identically everywhere.

> **Rules should map to real attacker behavior, not just "unusual."** The strongest correlation rules are built directly from known attack techniques — frameworks like MITRE ATT&CK (see the Windows Event Log Reference guide, Section 7, for a direct application of this) provide a structured, well-documented source of exactly what patterns to actually look for.

// SECTION 6

Log Normalization & Parsing

6.1 THE PROBLEM: EVERY SOURCE SPEAKS DIFFERENTLY

A firewall, a Linux `auth.log` entry, and a Windows Event Log record all describe events using completely different formats, field names, and structures — even for conceptually identical events like "a login failed." Without normalization, writing a correlation rule (Section 5) that needs to consider all three sources would require separate, duplicated logic for every single format.

6.2 PARSING TO A COMMON SCHEMA

Normalization parses each source's raw log format into a shared, common schema — mapping each source's own field names onto standardized ones (e.g., every source's version of "who logged in" becomes a single `user` field, regardless of whether the original log called it `username`, `acct`, or `TargetUserName`). Once normalized, a single correlation rule can operate identically across every source that's been mapped to that schema.

SOURCE	RAW FIELD NAME	NORMALIZED FIELD
Linux auth.log	(embedded in free-text message)	`user`
Windows Event Log	`TargetUserName`	`user`
Cloud IAM log	`principalId`	`user`

6.3 PARSERS, DECODERS & THE FRAGILITY PROBLEM

The components that perform this parsing (called decoders in Wazuh, or props/transforms in Splunk) are typically written against a *specific* expected log format. A source application updating its log format — even a seemingly minor change — can silently break the parser, causing that source's events to arrive unnormalized or entirely unparsed. This is a genuinely common, easy-to-miss failure mode: alerts quietly stop firing not because nothing suspicious happened, but because the pipeline stopped understanding the incoming data at all.

> **Silent parsing failures are a real detection gap.** Monitoring the health of the parsing/normalization stage itself — not just the alerts it eventually produces — is a genuine, ongoing SOC responsibility, not a one-time setup task. An unparsed log source is, for detection purposes, effectively an unmonitored one.

// SECTION 7

Alert Tuning & False Positive Management

7.1 THE ALERT FATIGUE PROBLEM

A correlation rule (Section 5) that's too broad generates a flood of alerts for entirely benign activity — **false positives**. Beyond wasting analyst time on each individual false alarm, high false-positive volume causes **alert fatigue**: analysts start reflexively dismissing alerts from a noisy rule without properly investigating, which is exactly the condition under which a genuine attack slips through unnoticed, indistinguishable from the surrounding noise.

7.2 TUNING TECHNIQUES

TECHNIQUE	HOW IT HELPS
Allow-listing known-good activity	Excludes recognized, legitimate patterns (a scheduled backup job, an admin's normal script) from triggering
Adjusting thresholds	Raising a "5 failed logins in 2 minutes" rule's numbers if normal activity is naturally noisier than expected
Adding context requirements	Requiring a rule to also check asset criticality or time-of-day, narrowing matches to genuinely unusual combinations
Severity tiering	Routing lower-confidence matches to a lower-priority queue instead of an identical high-urgency alert

7.3 THE TUNING TRADE-OFF

Every tuning change trades off against catching genuine positives — an allow-list rule too broad might exclude an attacker who's learned to mimic legitimate patterns; a threshold raised too high might miss a slower, more patient attack designed to stay under it. Tuning is never a one-time task; it's an ongoing balance, revisited as both the environment and the threat landscape change.

> **Track false positive rate as a real, ongoing metric — not just anecdotally.** A rule generating dozens of alerts a day that are consistently closed as benign is actively costing analyst time and contributing to fatigue, even if no one has explicitly complained about it yet. Regularly reviewing per-rule alert volume and disposition (true positive vs false positive) is what actually surfaces rules that need tuning before they quietly get ignored.

// SECTION 8

Worked Example: Building a Correlation Rule

A simplified, Wazuh-style rule detecting a classic pattern: repeated SSH authentication failures followed by a success — a strong brute-force indicator, pulling together concepts from Sections 2, 5, and 6.

8.1 THE BASE RULE: DETECTING FAILED SSH LOGINS

```
$ <rule id="100010" level="5">
$  <if_sid>5716</if_sid>
$  <match>Failed password</match>
$  <description>SSH authentication failed</description>
$ </rule>
```

This rule matches the normalized "failed password" pattern (Section 6.2) from the SSH decoder's parent rule ID 5716 — a single-event rule (Section 5.2) on its own.

8.2 THE THRESHOLD RULE: MULTIPLE FAILURES

```
$ <rule id="100011" level="10" frequency="5" timeframe="120">
$  <if_matched_sid>100010</if_matched_sid>
$  <description>Possible SSH brute force — 5 failures in 2 minutes</description>
$ </rule>
```

Builds on the base rule, requiring 5 matches within 120 seconds (Section 5.2's threshold-based pattern) before firing — a single failed login is normal and expected; five in two minutes is not.

8.3 THE CORRELATION RULE: FAILURES THEN SUCCESS

```
$ <rule id="100012" level="12">
$  <if_matched_sid>100011</if_matched_sid>
$  <match>Accepted password</match>
$  <description>SSH brute force followed by successful login</description>
$ </rule>
```

The highest-severity rule: it only fires if the brute-force pattern (rule 100011) was already matched, *and* a subsequent successful login follows — the multi-event correlation (Section 5.2) that turns "someone's guessing passwords" into "someone guessed correctly and got in," a dramatically more urgent finding worth immediate response.

> **Notice the layered structure.** Each rule builds on the previous one rather than duplicating logic — exactly the kind of specific, purposeful escalation (Section 5.3) that produces a genuinely actionable high-severity alert, rather than a flood of low-value single-failed-login noise that would quickly contribute to alert fatigue (Section 7.1).

// SECTION 9

Quick Reference & Glossary

TERM	DEFINITION
SIEM	Security Information and Event Management — a platform for centralized log collection, correlation, and alerting.
Agent / Forwarder	Lightweight software on a monitored host that collects and sends log data to the SIEM.
Normalization	Parsing different log formats into a shared, common schema.
Enrichment	Adding extra context (geolocation, threat intel matches) to an event before correlation.
Correlation Rule	Logic defining a pattern of events that, when matched, generates an alert.
Threshold-Based Rule	A rule requiring a certain count of matching events within a time window before firing.
False Positive	An alert fired on benign activity that isn't actually malicious.
False Negative	Malicious activity that fails to trigger any alert.
Alert Fatigue	Analysts disengaging from alerts due to a high volume of low-value ones.
Wazuh	An open-source SIEM/XDR platform with built-in host-level monitoring capability.
Splunk	A widely deployed commercial SIEM platform using SPL as its query language.

End of guide. A SIEM is only as good as the pipeline feeding it — well-normalized data, well-tuned rules, and honestly-tracked false positive rates matter more than which specific product name sits at the center. The five-stage pipeline from Section 2 is the mental model that transfers between any platform you'll actually work with.